

Training loss down, validation loss up

Give a big network eighty noisy points and watch it memorise them. Then fix it three ways — a weight penalty, stopping early, and a smaller network — and compare the evidence side by side.

Scaffolded lab

22 minutes

scikit-learn

Three fixes

Before you start

Setup · given

1 · overfit on purpose · ...

2 · fix 1: an L2 penalty

3 · fix 2: stop early

4 · fix 3: less capacity

5 · one figure, one table · ...

Reflection

If you finish early

Before you start

Open the provided `lab15.ipynb`. The data and a `run` helper that records training and validation loss after every epoch are given. Everything runs on a laptop CPU in under a minute.



By the end of this lab you can

- Recognise overfitting from training and validation curves.
- Reduce it with an L2 penalty, with early stopping, and with less capacity.
- Compare the three fixes on the same evidence, in a table.

LAB 15

Setup · given

Two interleaved half-moons with noise: 80 points to train on, 80 to validate on. Before running anything, **sketch** the two loss curves you expect for a network with two hidden layers of 128 units.

```

import copy, warnings
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import log_loss
warnings.filterwarnings("ignore")

X, y = make_moons(n_samples=160, noise=0.30, random_state=7)
X_tr, X_va, y_tr, y_va = train_test_split(X, y, test_size=0.5, random_state=7, stratify=y)

def run(hidden=(128, 128), alpha=0.0, epochs=400, X_train=None, y_train=None):
    """Train one epoch at a time; record both losses; remember the model at its best
    validation epoch."""
    X_train = X_tr if X_train is None else X_train
    y_train = y_tr if y_train is None else y_train
    clf = MLPClassifier(hidden_layer_sizes=hidden, alpha=alpha, learning_rate_init=0.01,
random_state=7)
    train_loss, val_loss, best = [], [], (np.inf, None, 0)
    for epoch in range(epochs):
        clf.partial_fit(X_train, y_train, classes=[0, 1])
        train_loss.append(log_loss(y_train, clf.predict_proba(X_train)))
        val_loss.append(log_loss(y_va, clf.predict_proba(X_va)))
        if val_loss[-1] < best[0]:
            best = (val_loss[-1], copy.deepcopy(clf), epoch)
    return clf, np.array(train_loss), np.array(val_loss), best

def summary(name, clf, val_loss, best):
    print(f"{name:<16} final val loss {val_loss[-1]:.3f}    best {best[0]:.3f} at epoch
{best[2]:<4} "
        f"val accuracy: final {clf.score(X_va, y_va):.3f}, at best epoch
{best[1].score(X_va, y_va):.3f}")

```

TASK 1 • 4 MINUTES

overfit on purpose • given

1

Run it and compare the plot with your sketch.

given

PYTHON

```
base, tr0, va0, best0 = run()
plt.plot(tr0, label="training"); plt.plot(va0, label="validation")
plt.axvline(best0[2], color="grey", linestyle="--", label=f"best validation epoch ({best0[2]})")
plt.xlabel("epoch"); plt.ylabel("log loss"); plt.legend(); plt.title("A big network on 80 points")
plt.show()
summary("no fix", base, va0, best0)
```

what it should print

OUTPUT

```
no fix          final val loss 0.881    best 0.121 at epoch 38    val accuracy: final 0.912,
at best epoch 0.963
```



 **The gap is the evidence.** After the best epoch, training loss keeps falling and validation loss climbs: from then on the network is learning *these* 80 points, noise included, rather than the moons.

TASK 2 · 5 MINUTES

fix 1: an L2 penalty

2

`alpha` adds a cost for large weights to the loss. Try a few values between 0.01 and 1 and keep the one with the lowest **final** validation loss.

your turn

PYTHON

```
alpha_choice = None          # TODO: your chosen alpha
l2, tr1, va1, best1 = run(alpha=alpha_choice)
summary(f"L2 alpha={alpha_choice}", l2, va1, best1)
assert va1[-1] < va0[-1], "the penalty should end with a lower validation loss than no fix"
```

one version that works

PYTHON

```
alpha_choice = 0.5
l2, tr1, va1, best1 = run(alpha=alpha_choice)
summary(f"L2 alpha={alpha_choice}", l2, va1, best1)
assert va1[-1] < va0[-1]
```

what it should print

OUTPUT

```
L2 alpha=0.5      final val loss 0.146    best 0.142 at epoch 308  val accuracy: final
0.963, at best epoch 0.950
```

TASK 3 · 4 MINUTES

fix 2: stop early

3

No retraining needed. The `run` helper already kept a copy of the model at its best validation epoch in `best0[1]`. Report that model.

your turn

PYTHON

```
early_model = None      # TODO: the model saved at the best validation epoch of the first
run
early_epoch = None      # TODO: which epoch that was
print("stopped at epoch", early_epoch, "  val accuracy", round(early_model.score(X_va, y_va),
3),
      "  instead of", round(base.score(X_va, y_va), 3), "after", len(va0), "epochs")
```

one version that works

PYTHON

```
early_model = best0[1]
early_epoch = best0[2]
print("stopped at epoch", early_epoch, " val accuracy", round(early_model.score(X_va,
y_va), 3),
      " instead of", round(base.score(X_va, y_va), 3), "after", len(va0), "epochs")
```

what it should print

OUTPUT

```
stopped at epoch 38  val accuracy 0.963  instead of 0.912 after 400 epochs
```

TASK 4 · 4 MINUTES

fix 3: less capacity

4

Choose a much smaller network — one hidden layer, a handful of units — and rerun with no penalty.

your turn

PYTHON

```
small_hidden = None      # TODO: a tuple, e.g. (n,)
small, tr3, va3, best3 = run(hidden=small_hidden)
summary(f"hidden={small_hidden}", small, va3, best3)
assert va3[-1] < va0[-1]
```

one version that works

PYTHON

```
small_hidden = (4,)
small, tr3, va3, best3 = run(hidden=small_hidden)
summary(f"hidden={small_hidden}", small, va3, best3)
assert va3[-1] < va0[-1]
```

what it should print

OUTPUT

```
hidden=(4,)      final val loss 0.255    best 0.255 at epoch 325  val accuracy: final
0.912, at best epoch 0.925
```

TASK 5 · 5 MINUTES

one figure, one table · plotting given

5

given

PYTHON

```
for curve, name in [(va0, "no fix"), (va1, "L2"), (va3, "smaller network")]:
    plt.plot(curve, label=name)
plt.axvline(best0[2], color="grey", linestyle="--", label="early stop")
plt.xlabel("epoch"); plt.ylabel("validation log loss"); plt.legend(); plt.title("Three fixes,
one figure")
plt.show()
```

Intervention	Final validation loss	Validation accuracy	What changed in the curve?
None			
L2			
Early stopping			
Smaller network			

Reflection



Write it in the notebook, in your own words

All three fixes reduce the gap. Which one would you reach for **first** on a new problem, and what would make you choose a different one?

Your answer:

If you finish early

A. More data beats more cleverness. Generate 1,000 training points with `make_moons(n_samples=1080, noise=0.30, random_state=7)`, keep the same 80 validation points, and rerun the original big network with no fix (pass `X_train=` and `y_train=` to `run`). Does the gap shrink?

B. Session 22's lecture showed dropout. scikit-learn does not offer it; in PyTorch it is one layer, `nn.Dropout(0.35)`. Where in the network would you put it?

your turn

PYTHON

```
# TODO (optional extension A)
```